

Lineare Suche – Index im Integer-Array

Pseudocode (IHK-Stil):

```
funktion lineareSuche(A: Array<Integer>, x: Integer) → Integer
    für i = 0 bis länge(A)-1
        wenn A[i] == x dann
            return i
        end wenn
    end für
    return -1
end funktion
```

Java-Beispiel:

```
public class LineareSucheDemo {
    static int lineareSuche(int[] A, int x) {
        for (int i = 0; i < A.length; i++) {
            if (A[i] == x) return i;
        }
        return -1;
    }
}
```

Lineare Suche – Erklärung & Beispiele

Prüft alle Elemente von links nach rechts, bis ein Treffer gefunden ist oder das Ende erreicht wird.

****Voraussetzungen:**** keine (funktioniert auch bei unsortierten Daten).

****Komplexität:**** $O(n)$ Zeit, $O(1)$ Speicher.

****Prüfungstipp:**** Bei „erstes Vorkommen“ → sofort return bei Treffer (früher Abbruch).

Array	x	Rückgabe (Index)
[4,7,2,7]	7	1
[1,3,5]	2	-1
[9]	9	0

Binäre Suche – Index im sortierten Integer-Array

Pseudocode (IHK-Stil):

```
funktion binaerSuche(A: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(A) - 1
    solange links ≤ rechts
        mitte = (links + rechts) / 2
        wenn A[mitte] == x dann
            return mitte
        sonst wenn A[mitte] < x dann
            links = mitte + 1
        sonst
            rechts = mitte - 1
    end solange
    return -1
end funktion
```

Java-Beispiel:

```
public class BinaereSucheDemo {
    static int binaerSuche(int[] A, int x) {
        int l = 0, r = A.length - 1;
        while (l <= r) {
            int m = l + (r - l) / 2; // overflow-sicher
            if (A[m] == x) return m;
            else if (A[m] < x) l = m + 1;
            else r = m - 1;
        }
        return -1;
    }
}
```

Binäre Suche – Erklärung & Beispiele

Teile-&-Herrsche: In jeder Iteration wird die Hälfte ausgeschlossen.

****Voraussetzung:**** Array ****muss sortiert**** sein (aufsteigend in diesem Beispiel).

****Komplexität:**** $O(\log n)$ Zeit, $O(1)$ Speicher.

****Typische Fehler:**** Mitte falsch berechnet; nicht sortierte Daten; Endlosschleife bei falschen Grenzen.

****Prüfungstipp:**** In Java Overflow vermeiden: $m = l + (r - l) / 2$.

Array (sortiert)	x	Rückgabe (Index)
[1,3,5,7,9]	7	3
[2,4,6,8]	1	-1

Binäre Suche – Kurs nach Kursnummer (Objekt zurück)

Pseudocode (IHK-Stil):

```
funktion sucheKurs(kurse: Array<Kurs>, nr: Integer) → Kurs
    links = 0
    rechts = länge(kurse) - 1
    solange links ≤ rechts
        mitte = (links + rechts) / 2
        if kurse[mitte].getKursnummer() == nr dann
            return kurse[mitte]
        else if kurse[mitte].getKursnummer() < nr dann
            links = mitte + 1
        else
            rechts = mitte - 1
    end solange
    return null
end funktion
```

Java-Beispiel:

```
class Kurs {
    private int kursnummer;
    public Kurs(int nr){ this.kursnummer = nr; }
    public int getKursnummer(){ return kursnummer; }
}

public class SucheKursDemo {
    static Kurs sucheKurs(Kurs[] kurse, int nr) {
        int l = 0, r = kurse.length - 1;
        while (l <= r) {
            int m = l + (r - l) / 2;
            int km = kurse[m].getKursnummer();
            if (km == nr) return kurse[m];
            else if (km < nr) l = m + 1;
            else r = m - 1;
        }
        return null;
    }
}
```

Binäre Suche – Objektzugriff – Erklärung & Beispiele

Gleiche Logik wie bei Zahlen, aber der **Vergleich** erfolgt über einen Schlüssel (hier: ``getKursnummer()``).

Voraussetzung: ``kurse`` ist **nach Kursnummer sortiert**.

Variante in IHK: Rückgabe kann Index **oder** Objekt sein – Aufgabenstellung genau lesen.

Kursnummern	Gesucht	Ergebnis
[10,20,30,40]	30	Kurs(30)
[5,15,25]	7	null

Lineare Suche – String (case-insensitive)

Pseudocode (IHK-Stil):

```
funktion findeName(L: Liste<String>, s: String) → Integer
    such = lower(s)
    für i = 0 bis L.size() - 1
        wenn lower(L.get(i)) == such dann
            return i
        end wenn
    end für
    return -1
end funktion
```

Java-Beispiel:

```
import java.util.List;

public class FindeNameDemo {
    static int findeName(List<String> L, String s) {
        String such = s.toLowerCase();
        for (int i = 0; i < L.size(); i++) {
            if (L.get(i).toLowerCase().equals(such)) return i;
        }
        return -1;
    }
}
```

Lineare Suche – Strings – Erklärung & Beispiele

Vergleich von Strings häufig **fallunabhängig** (case-insensitive).

Typische Fehler: ``==`` statt ``equals()`` in Java; ``toLowerCase()`` vergessen; Nullwerte nicht beachtet.

Prüfungstipp: Bei Internationalisierung ggf. ``toLowerCase(Locale)`` erwähnen.

Liste	x	Rückgabe (Index)
["Anna", "Bernd", "lisa"]	"LISA"	2
["Tom", "Tim"]	"Max"	-1

Binäre Suche – generisch mit Key-Funktion

Pseudocode (IHK-Stil):

```
funktion binaerSucheKey(L: Liste<T>, key: Funktion<T→Integer>, such: Integer)
    l = 0
    r = L.size() - 1
    solange l ≤ r
        m = (l + r) / 2
        k = key(L.get(m))
        wenn k == such dann return m
        sonst wenn k < such dann l = m + 1
        sonst r = m - 1
    end solange
    return -1
end funktion
```

Java-Beispiel:

```
import java.util.List;
import java.util.function.Function;

public class BinaerSucheKeyDemo {
    static <T> int binaerSucheKey(List<T> L, Function<T,Integer> key, int such) {
        int l = 0, r = L.size()-1;
        while (l <= r) {
            int m = l + (r - l) / 2;
            int k = key.apply(L.get(m));
            if (k == such) return m;
            else if (k < such) l = m + 1;
            else r = m - 1;
        }
        return -1;
    }
}
```

Generische Binärsuche – Erklärung & Beispiele

Verallgemeinerung für beliebige Objekte: Die **Key-Funktion** liefert den Sortierschlüssel (z. B. ``getKundenNr()``).

Voraussetzung: Liste ist nach ``key`` **sortiert**.

Prüfungstipp: In IHK-Pseudocode reicht oft die Angabe „key(T) liefert int“.

Objekte (key)	such	Rückgabe
Kunden Nr=[10,20,30]	20	Index 1
Artikel Preis=[5,9,12]	7	-1